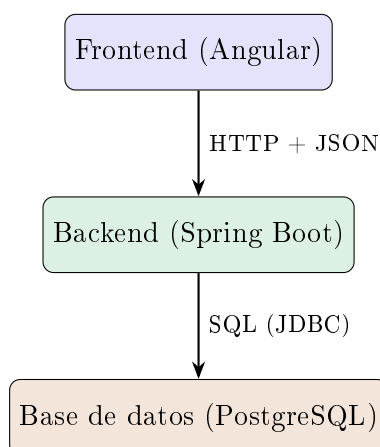


Gestor de Tareas (To – Do)

Manual explicado desde cero

«Tutorial para MC Burro» — entiende tu propio proyecto



Aplicación Fullstack: Spring Boot 3 + Angular 18 + PostgreSQL

30 de junio de 2026

Índice

1. Antes de empezar: las 3 piezas	2
2. El panorama general	2
3. ¿Qué es una API REST? (HTTP y JSON sin dolor)	2
4. El backend por dentro (Spring Boot)	3
4.1. Las capas: una cadena de montaje	3
4.2. Entidades y DTOs: dos cosas distintas	4
4.3. Validación automática	4
4.4. Manejo de errores en un solo sitio	4
4.5. Seguridad: contraseñas y JWT	5
4.6. Flyway: el historial de la base de datos	5
5. El frontend por dentro (Angular)	5
5.1. Componentes: las pantallas	5
5.2. Servicios: el que hace las llamadas	5
5.3. El interceptor: pone la «pulsera» en cada petición	6
5.4. El guard: el portero de las rutas	6
5.5. Reactive Forms y el filtro de estado	6
6. ¿Cómo se comunican? El viaje de una petición	6
6.1. El flujo de login con JWT	7
7. La base de datos (PostgreSQL)	7
8. ¿Qué es eso de Docker?	7
8.1. docker-compose: varios contenedores a la vez	8
9. Cómo se ejecuta todo	8
10. Glosario rápido	9

1. Antes de empezar: las 3 piezas

Tu aplicación está formada por **tres programas** que trabajan juntos. Si entiendes qué hace cada uno, ya entendiste el 80 % del proyecto.

1. **El frontend** (Angular): es lo que el usuario *ve* en el navegador (la pantalla de login, la lista de tareas, los botones).
2. **El backend** (Spring Boot): es el «cerebro». Recibe peticiones, decide qué se puede hacer, aplica las reglas y habla con la base de datos.
3. **La base de datos** (PostgreSQL): es el «almacén» donde se guardan los datos de forma permanente (usuarios y tareas), aunque apagues el equipo.

Analogía

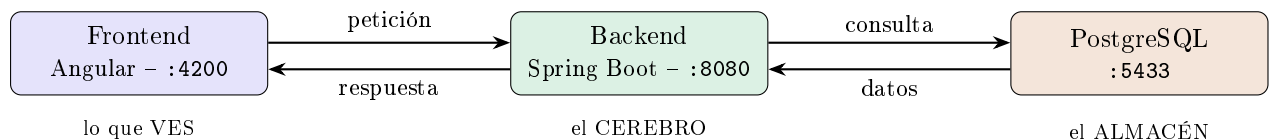
Imagina un **restaurante**:

- El **comedor y el camarero** = el **frontend**. Es lo que tú, cliente, ves y tocas. El camarero toma tu pedido y te trae los platos.
- La **cocina** = el **backend**. Ahí se decide qué se puede cocinar, se siguen las recetas (las reglas) y se preparan los platos.
- La **despensa/almacén** = la **base de datos**. Guarda los ingredientes (los datos). La cocina va a la despensa a coger o guardar cosas.

El cliente *nunca* entra a la cocina ni a la despensa: sólo habla con el camarero. Igual que el navegador nunca habla directamente con la base de datos: siempre pasa por el backend.

2. El panorama general

Cuando usas la app, la información viaja así:



Cada pieza corre en su propio «puerto» (una especie de número de puerta del equipo): el frontend en el 4200, el backend en el 8080 y la base de datos en el 5433.

3. ¿Qué es una API REST? (HTTP y JSON sin dolor)

El frontend y el backend hablan por **HTTP**, el mismo protocolo que usa el navegador para abrir páginas web. Una **API REST** es, simplemente, una lista de «direcciones» (URLs) a las que el frontend puede mandar peticiones para pedir o cambiar datos.

Cada petición usa un **verbo** que indica la intención:

Verbo HTTP	Significado
GET	«Dame información» (leer)
POST	«Crea algo nuevo»
PUT	«Actualiza algo que ya existe»
DELETE	«Borra algo»

El backend responde con un **código de estado** que dice cómo fue:

Código	Significado
200 / 201	Todo bien (201 = «creado»)
400	Mandaste datos mal (validación)
401	No estás identificado (falta token o caducó)
404	No existe lo que pides
409	Conflicto (p. ej. usuario ya registrado)

Los datos viajan en formato **JSON**, que es texto fácil de leer. Por ejemplo, una tarea se ve así:

```
{
  "id": 1,
  "title": "Comprar pan",
  "status": "PENDIENTE",
  "dueDate": "2026-07-15"
}
```

Analogía

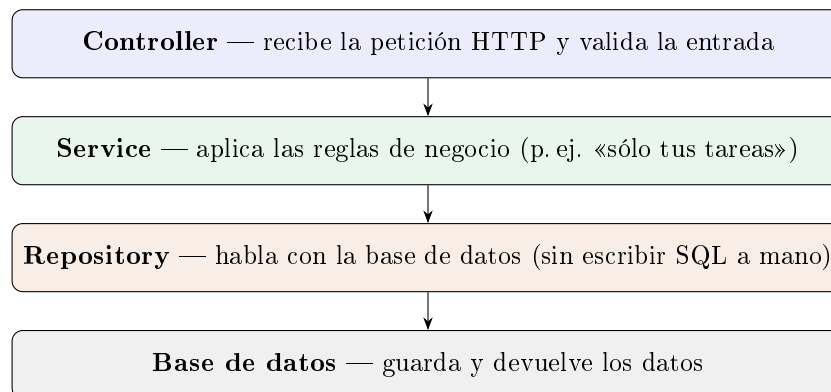
JSON es como la **nota de pedido** del camarero: un papelito con un formato acordado («1 café, 2 tostadas») que tanto el comedor como la cocina entienden.

4. El backend por dentro (Spring Boot)

Spring Boot es un framework de Java que te da «hecho» todo lo aburrido (servidor web, conexión a la base de datos, seguridad...) para que tú sólo escribas la lógica de tu app.

4.1 Las capas: una cadena de montaje

El backend está organizado en **capas**, y una petición las atraviesa en orden. Esto mantiene el código ordenado: cada capa tiene un único trabajo.



Analogía

En la cocina: el **Controller** es el *pase* (donde se reciben las comandas), el **Service** es el *chef* (sabe las recetas y las reglas), y el **Repository** es el *ayudante* que va a la despensa a por los ingredientes.

Así se ve el Controller de tareas (archivo `TaskController.java`). Fíjate que es muy «fino»: sólo recibe y delega.

```
@RestController
@RequestMapping("/api/tasks") // todas las rutas empiezan por /api/tasks
public class TaskController {
```

```

private final TaskService taskService;    // delega en el Service

// GET /api/tasks -> lista las tareas del usuario autenticado
@GetMapping
public List<TaskResponse> list(@AuthenticationPrincipal UserDetails user,
                               @RequestParam(required = false) TaskStatus
                               status) {
    return taskService.list(user.getUsername(), status);
}

// POST /api/tasks -> crea una tarea
@PostMapping
public ResponseEntity<TaskResponse> create(@AuthenticationPrincipal
                                             UserDetails user,
                                             @Valid @RequestBody TaskRequest
                                             request) {
    return ResponseEntity.status(201).body(taskService.create(user.getUsername()
                                                                (), request));
}
}

```

4.2 Entidades y DTOs: dos cosas distintas

- Una **Entidad** (`Task.java`, `User.java`) representa una *tabla* de la base de datos. Es la forma «interna» de los datos.
- Un **DTO** (*Data Transfer Object*) es la forma «pública» que viaja por HTTP. Por ejemplo `TaskRequest` (lo que entra) y `TaskResponse` (lo que sale).

Nota

¿Por qué separarlos? Porque la entidad `User` guarda la **contraseña cifrada**, y eso *jamás* debe salir hacia el navegador. Usando DTOs decides exactamente qué se muestra y qué no.

4.3 Validación automática

En los DTOs se ponen «etiquetas» que comprueban los datos antes de procesarlos:

```

public record RegisterRequest(
    @NotBlank @Size(min = 3, max = 50) String username,    // obligatorio, 3-50
    // letras
    @NotBlank @Size(min = 6) String password              // obligatorio, 6+
    // letras
) {}

```

Si el usuario manda una contraseña de 3 letras, el backend responde 400 con un mensaje claro, *sin* llegar siquiera a la base de datos.

4.4 Manejo de errores en un solo sitio

En vez de repetir `try/catch` por todos lados, hay una clase (`GlobalExceptionHandler`) marcada con `@RestControllerAdvice` que «atrapa» cualquier error y lo convierte en una respuesta JSON uniforme con el código correcto (400, 401, 404, 409...). Resultado: errores consistentes y controllers limpios.

4.5 Seguridad: contraseñas y JWT

Dos ideas clave:

1. Las **contraseñas se cifran** con un algoritmo llamado **BCrypt** antes de guardarlas. En la base de datos no hay «secreto123», sino algo como `$2a$10$N9qo8uL...` (irreversible). Aunque alguien robe la base de datos, no ve las contraseñas reales.
2. El **token JWT** es un «pase» que el backend te entrega al iniciar sesión. En cada petición siguiente lo enseñas para demostrar quién eres, sin tener que reenviar tu contraseña.

Analogía

El JWT es como la **pulsera de una discoteca**: enseñas tu DNI *una vez* en la entrada (login) y te ponen una pulsera (el token). Después, para entrar y salir, basta con enseñar la pulsera. El portero la mira y, si es auténtica y no ha caducado, te deja pasar.

Nota

El token va firmado con una clave secreta que sólo conoce el servidor. Si alguien intenta falsificarlo, la firma no cuadra y el backend lo rechaza con un **401**.

4.6 Flyway: el historial de la base de datos

¿Cómo se crean las tablas? Con **Flyway**. Son archivos SQL numerados (`V1__create_users_table.sql`, `V2__create_tasks_table.sql`) que se ejecutan **automáticamente** la primera vez que arranca el backend.

Analogía

Flyway es como el **libro de recetas con fecha** de la cocina: «paso 1, montamos esta estancia; paso 2, añadimos esta otra». Cualquiera que abra el restaurante en otro local obtiene exactamente la misma cocina, en el mismo orden.

5. El frontend por dentro (Angular)

Angular es un framework para construir la parte visual. La app es una *SPA* (Single Page Application): el navegador carga una vez y luego cambia de pantalla sin recargar, pidiendo sólo los datos al backend.

5.1 Componentes: las pantallas

Un **componente** es una pieza de interfaz. Cada uno tiene 3 archivos:

<code>.ts</code>	la lógica (qué pasa al pulsar un botón)
<code>.html</code>	lo que se ve (la estructura visual)
<code>.css</code>	cómo se ve (colores, tamaños)

En el proyecto hay dos pantallas: `login.component` (login/registro) y `dashboard.component` (la lista de tareas con el menú lateral).

5.2 Servicios: el que hace las llamadas

La lógica de *hablar con el backend* no vive en los componentes, sino en **servicios** (`auth.service.ts`, `task.service.ts`). Así el componente sólo dice «créame esta tarea» y el servicio se encarga del

HTTP.

```
@Injectable({ providedIn: 'root' })
export class TaskService {
  private http = inject(HttpClient);
  private baseUrl = 'http://localhost:8080/api/tasks';

  // Devuelve las tareas; si se pasa un estado, filtra por él.
  list(status?: TaskStatus | null) {
    let params = new HttpParams();
    if (status) params = params.set('status', status);
    return this.http.get<Task[]>(this.baseUrl, { params });
  }

  create(task: TaskRequest) { return this.http.post<Task>(this.baseUrl, task);
  }
}
```

5.3 El interceptor: pone la «pulsera» en cada petición

El **interceptor** (`auth.interceptor.ts`) es un «portero» que intercepta *todas* las peticiones salientes y les añade automáticamente el token JWT. Así ningún servicio tiene que acordarse de hacerlo.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  const authReq = token
    ? req.clone({ setHeaders: { Authorization: 'Bearer ${token}' } })
    : req;
  return next(authReq); // continúa la petición, ya con el token puesto
};
```

5.4 El guard: el portero de las rutas

El **guard** (`auth.guard.ts`) protege la ruta del dashboard: si no hay sesión, te manda al login. Es lo que impide entrar al tablero escribiendo la URL a mano sin haber iniciado sesión.

5.5 Reactive Forms y el filtro de estado

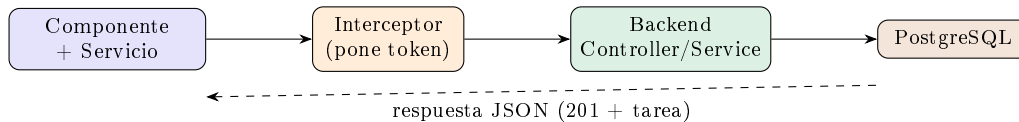
Los formularios (login y crear/editar tarea) usan **Reactive Forms**: el formulario se define en el código TypeScript, lo que facilita validarlo. El **filtro por estado** del menú lateral también es un control reactivo: cada vez que eliges «Pendientes», «Completadas», etc., su valor cambia y la lista se recarga pidiendo al backend sólo esas tareas.

6. ¿Cómo se comunican? El viaje de una petición

Juntemos todo. Esto es lo que pasa, paso a paso, cuando **creas una tarea**:

1. Pulsas «Guardar» en el navegador. El **componente** llama a `TaskService.create(...)`.
2. El **servicio** hace un POST a `http://localhost:8080/api/tasks` con la tarea en JSON.
3. El **interceptor** añade la cabecera `Authorization: Bearer <token>`.
4. Llegas al **backend**. Un filtro de seguridad valida el token y sabe quién eres.
5. El **Controller** recibe la petición, valida los datos y llama al **Service**.

6. El **Service** crea la tarea asociándola a *tu* usuario y pide al **Repository** que la guarde.
7. El **Repository** hace un **INSERT** en **PostgreSQL**.
8. El backend responde 201 con la tarea creada (en JSON).
9. El frontend recibe la respuesta y refresca la lista en pantalla.



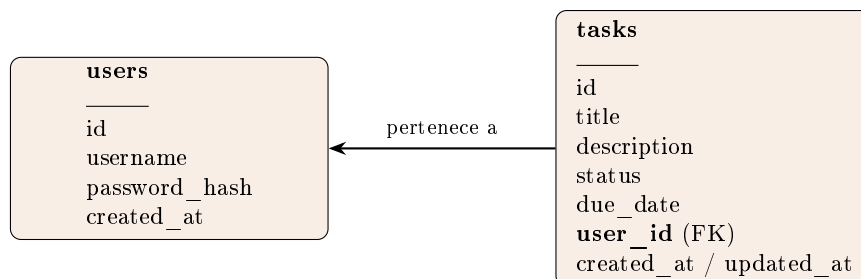
6.1 El flujo de login con JWT

1. Mandas usuario y contraseña a `/api/auth/login`.
2. El backend comprueba la contraseña (BCrypt) y, si es correcta, te devuelve un **token JWT**.
3. El frontend guarda el token (en el *localStorage* del navegador).
4. En cada petición, el interceptor envía ese token.
5. Si el token caduca, el backend responde 401 y el frontend te devuelve al login automáticamente.

7. La base de datos (PostgreSQL)

PostgreSQL es una base de datos *relacional*: guarda la información en **tablas** (como hojas de cálculo), con **filas** (cada registro) y **columnas** (cada dato).

Nuestro proyecto tiene dos tablas:



La columna `user_id` de `tasks` es una **clave foránea** (FK): un «puntero» que dice a qué usuario pertenece cada tarea. Gracias a ella, el backend puede pedir «todas las tareas *de este usuario*» y nunca mezclar las de unos con las de otros.

Nota

El backend se conecta a PostgreSQL mediante **JDBC** (el «cable» estándar de Java para bases de datos). La dirección, el usuario y la contraseña están en `application.yml`. Tú no escribes SQL a mano: Spring Data JPA traduce métodos como `findByOwnerIdAndStatus(...)` a consultas SQL por ti.

8. ¿Qué es eso de Docker?

El problema que resuelve. ¿Te ha pasado que algo «funciona en mi máquina» pero no en otra? Eso ocurre porque cada equipo tiene versiones distintas de programas. **Docker** resuelve

esto empaquetando un programa *junto con todo lo que necesita* para funcionar, de modo que corra igual en cualquier sitio.

Imagen una «plantilla» congelada con un programa y sus dependencias (por ejemplo, «PostgreSQL 17 listo para usar»).

Contenedor una imagen *en ejecución*. Puedes arrancar, parar y borrar contenedores sin ensuciar tu equipo.

Analogía

Una imagen Docker es como un **contenedor de mudanzas sellado**: dentro va todo lo necesario ya montado. Da igual el camión (tu equipo) que lo transporte: al abrirlo, todo está exactamente igual. Un contenedor es ese mismo cajón ya «abierto y funcionando».

8.1 docker-compose: varios contenedores a la vez

En vez de arrancar cada contenedor a mano, el archivo `docker-compose.yml` describe *qué* servicios quieres y los levanta con un comando. El nuestro define dos: la base de datos (`db`) y la API (`api`).

```
services:
  db:
    image: postgres:17-alpine          # contenedor de PostgreSQL
    ports: ["5433:5432"]               # puerto 5433 de tu equipo -> 5432 del
      contenedor
  api:
    build: ./backend                  # contenedor del backend
    depends_on: [db]                 # se construye con el Dockerfile del backend
    ports: ["8080:8080"]              # espera a que la base de datos esté lista
```

Con `docker compose up -d db` levantas sólo la base de datos; con `docker compose up --build` levantas base de datos + backend.

¡Ojo!

En este equipo no hay Docker instalado. Por eso, para poder ejecutar el proyecto, instalamos un **PostgreSQL «portable»** en tu carpeta personal (`~/local`), sin permisos de administrador. Es el *mismo* PostgreSQL real, sólo que arrancado sin Docker. Se gestiona con el comando `todo-pg` (`start`, `stop`, `status`). El `docker-compose.yml` sigue ahí y funcionará el día que instales Docker (y es uno de los entregables pedidos).

9. Cómo se ejecuta todo

Hacen falta las tres piezas a la vez. En este equipo:

```
# 1) Base de datos (PostgreSQL local, ya configurado)
todo-pg start                      # arranca en el puerto 5433

# 2) Backend (en una terminal)
cd backend && mvn spring-boot:run  # API en http://localhost:8080

# 3) Frontend (en otra terminal)
cd frontend && npm start             # UI en http://localhost:4200
```

Luego abre **http://localhost:4200** en el navegador. Para explorar la API directamente, entra a **http://localhost:8080/swagger-ui.html**.

10. Glosario rápido

API REST

Conjunto de URLs del backend a las que el frontend pide o cambia datos por HTTP.

Endpoint

Una de esas URLs concretas (p.ej. `/api/tasks`).

HTTP

El protocolo de comunicación de la web (verbos GET, POST...).

JSON

Formato de texto para intercambiar datos.

JWT

Token firmado que demuestra tu identidad sin reenviar la contraseña.

BCrypt

Algoritmo para cifrar contraseñas de forma irreversible.

Entidad

Clase Java que representa una tabla de la base de datos.

DTO

Objeto que define qué datos entran/salen por la API.

ORM / JPA

Tecnología que traduce objetos Java a filas de la base de datos (y al revés), evitando escribir SQL a mano.

Flyway

Herramienta que crea/versiona el esquema de la base de datos con scripts SQL numerados.

SPA

Aplicación web que carga una vez y cambia de pantalla sin recargar.

Componente

Pieza visual de Angular (lógica + HTML + CSS).

Servicio

Clase de Angular que centraliza la lógica (p.ej. llamadas HTTP).

Guard

«Portero» que decide si se puede entrar a una ruta.

Interceptor

«Portero» que modifica las peticiones HTTP (aquí, añade el token).

Docker

Tecnología para empaquetar programas con todo lo que necesitan.

Contenedor

Una imagen Docker en ejecución.

¡Y eso es todo! Si entiendes el viaje de una petición (Sección 7), ya puedes explicar el proyecto entero.